

Prolog Program to Check if a Bird Can Fly or Not

1. Aim

To write and execute a Prolog program to determine whether a particular bird can fly, using facts, rules, and negation as failure, and to incorporate the required queries.

2. Objective

- To understand how exceptions to a general rule are represented in Prolog.
- To use the cut (!) and negation (\+) to model default reasoning.
- To represent birds and flightless exceptions as facts.
- To retrieve whether a bird can fly using rules and queries.

3. Algorithm

1. Start the program.
2. Create a Prolog database using the predicate bird/1 to list all birds.
3. Create facts for flightless birds using the predicate flightless/1.
4. Define a rule can_fly/1: a bird can fly if it is a bird and not listed as flightless.
5. Enter the bird's name as a query.
6. Check whether the bird exists and whether it is flightless.
7. If the bird is not flightless, display that it can fly; otherwise display that it cannot fly.
8. Stop the program.

4. Flowchart



5. Code with Output

Code:

```
% -----  
% Bird flight database  
% -----  
bird(sparrow).  
bird(eagle).  
bird(penguin).  
bird(ostrich).  
bird(parrot).  
  
flightless(penguin).  
flightless(ostrich).  
  
% -----  
% Rule: a bird can fly unless it is flightless  
% -----  
can_fly(X) :-  
    bird(X),  
    \+ flightless(X).  
  
check_fly(X) :-  
    can_fly(X), !,  
    format("~w can fly~n", [X]).  
check_fly(X) :-  
    bird(X), !,  
    format("~w cannot fly~n", [X]).  
check_fly(X) :-  
    format("~w is not a known bird~n", [X]).
```

Output:

```
?- check_fly(sparrow).  
sparrow can fly  
true.  
  
?- check_fly(penguin).  
penguin cannot fly  
true.  
  
?- check_fly(dog).  
dog is not a known bird  
true.
```

6. Result

The Prolog program was successfully implemented to determine whether a given bird can fly, correctly handling both flying birds and flightless exceptions.

7. Conclusion

Thus, the bird-flight checking program was successfully implemented using Prolog. The experiment demonstrates the use of negation as failure ($\neg$) to model default rules with exceptions.